DATABASE DESIGN · VMS

Database Design Proposal

For review

The proposed tables, columns, keys, constraints and relationships for the VMS database.

PURPOSE	DETAIL LEVEL	STATUS
Present the proposed database design for review and discussion.	Columns, data types, keys, constraints and indexes for every table.	Revised proposal (v3), ideation phase. 34 tables; 76 foreign keys. Open decisions in Section 7.

DESIGN IDEA

Keep a person's identity separate from their login account, each visit and the pass used at the kiosk, so `people`, `users`, `visits` and `passes` are distinct tables. A visit is one record from request to close, whether pre-registered or walk-in, and `visit_events` keeps its history. The VMS serves our own organisation across one or more of its sites.

Keys, defaults, constraints and indexes shown below are proposed as database-level rules. Rules intended to be handled by the application instead are labelled **Application rule**.

1 Overall structure

The central records and how a visit is intended to flow through them.

PEOPLE
Person identity
May connect to a user or host

→

VISITS
Request to close
One per visitor per occasion

→

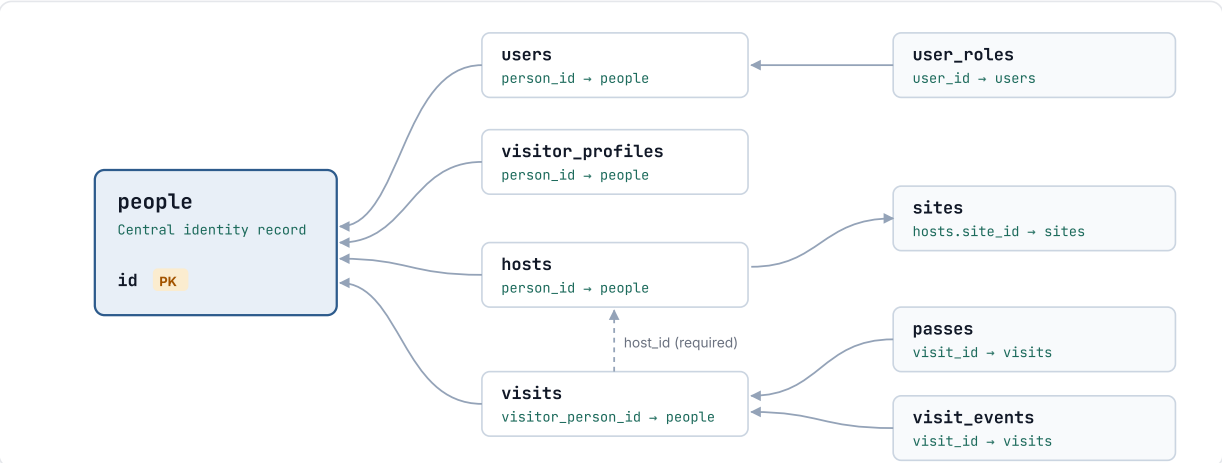
PASSES
QR credential
At most one per visit

→

VISIT_EVENTS
What happened
Append-only history

How the tables connect to `people`

Arrows point from the table holding the foreign key to the table it references. Only the main links are drawn; site, device and audit references are omitted.



1.1 Visit lifecycle

One `visits` row covers a visitor's occasion from request to close, whether pre-registered or walk-in (`source`). `status` holds where it is now, and `visit_events` records every step with who did it. Some situations are deliberately not statuses: an *overdue* visit is a checked-in visit past its end plus the site's grace period, shown as unconfirmed on the on-site list; an extension is a row in `visit_extensions`; marking a visit complete ends the pass's validity for entry, and the visit closes when the visitor checks out. Closed statuses set `closed_at`, which starts the retention clock.

STATUS	MEANING	ON SITE	CLOSED
`pending`	Waiting for a decision on the request: normally by the host (Section 1.2).	No	No
`security-review`	Held because screening found a possible blacklist match. No pass is issued and no check-in is possible until a security decision is recorded.	No	No
`scheduled`	Approved; the visitor is expected. A pass can be issued.	No	No
`checked-in`	The visitor has entered and is on site.	Yes	No
`temporarily-out`	The visitor has signed out but the visit stays open. Where the site allows re-entry and the pass is still valid, they can return on the same pass, including on later days of longer work.	No	No
`checked-out`	Final departure confirmed; the visit is closed.	No	Yes
`auto-closed`	Closed at the site's closing time without a confirmed departure. Kept apart from checked-out so it can be reviewed.	Unknown	Yes
`no-show`	Approved, but the visitor never arrived before the visit window ended.	No	Yes
`expired`	A request lapsed without a decision, for example a walk-in who left. A later approval is refused.	No	Yes
`cancelled`	Withdrawn by the visitor, host or reception before the visitor arrived.	No	Yes
`rejected`	Refused by the approver or by the security decision.	No	Yes

1.2 Walk-in identification and approval

A walk-in visitor is identified once, when they first register. Later entries and exits use the issued badge or QR and are checked without repeating OTP. Pre-registered visitors arrive with the pass from their approved request.

STEP	WHAT HAPPENS	RECORDED IN
1. Registration	The visitor, an assistant or a receptionist enters the visitor's details at the kiosk or the desk, and a walk-in visit is created in `pending`.	`visits` (source `walk-in`; `submitted_by_person_id` for an assistant)
2. Identification, once per visit	Normally a one-time code is sent to the visitor's phone or email. If OTP cannot be completed, for example with no phone, an authorised receptionist checks the visitor's ID, or issues a one-time code after checking it. A visitor without a phone is never simply let through.	`verification_codes`; `visits.verification_method`, `verified_at` and, for reception methods, `verified_by`. The database refuses to schedule a walk-in without them.

STEP	WHAT HAPPENS	RECORDED IN
3. Screening	The visitor is checked against restriction records; a possible match moves the visit to `security-review`.	`blacklist_matches`, `security_approvals` (Section 4.4)
4. Decision	The host normally approves or rejects. If the host does not respond, reception is alerted and the request expires after the site's limit. An authorised receptionist or an administrator may decide where the organisation's rules allow it. Never approved automatically, and never past an open security review.	`approved` or `rejected` event naming the person (database check)
5. Pass and badge	A pass is issued for the approved visit and the badge is printed.	`passes`; `pass_issued` event
6. Later entries and exits	The visitor scans their badge or QR. OTP is not asked for again: it identified the visitor at registration. Each scan still checks the pass, the visit status, restrictions and visiting hours.	`pass_scans`; `check_in`, `step_out`, `reentry`, `check_out` events

WHO	RESPONSIBILITY IN THE WALK-IN WORKFLOW
Host	Normally receives walk-in requests for their own visitors and approves or rejects them. Cannot decide a security review or override a restriction.
Authorised receptionist	Registers walk-ins at the desk and carries out the alternative identification when OTP cannot be completed. May decide a walk-in in circumstances the organisation defines (Section 7). Every such action names the receptionist.
Administrator	Handles escalations and exceptions within their sites: may approve, reject, or grant a recorded exception (`visit.override`). Cannot bypass a pending or denied security review.

1.3 Exceptional situations

How the proposed tables support the non-standard situations reception is expected to meet. Most need only a status, an event or a setting rather than a table of their own.

SITUATION	HOW THE DESIGN SUPPORTS IT	ENFORCED BY
Walk-in or unexpected personal visitor	A walk-in visit (category `personal` where relevant) follows Section 1.2.	Database values and checks; routing in the application
Host absent or not responding	After `pending_escalation_minutes` an `approval_escalated` event alerts reception. An authorised receptionist or an administrator may decide where the rules allow, or the visit is redirected (`host_changed`, with actor and reason). Never approved automatically.	Background job; database requires actor (and reason for a host change)
Visitor leaves before a decision	After `pending_expiry_minutes` the visit becomes `expired`; a later approval is refused and no pass exists. Reception or the visitor can cancel earlier.	Background job and application

SITUATION	HOW THE DESIGN SUPPORTS IT	ENFORCED BY
VIP while approval is pending	`priority` flags the request for faster handling; `submitted_by_person_id` records an assistant who registered on the VIP's behalf. Approval is still required.	Database columns; notification routing in the application
No smartphone, or OTP not received	Alternative identification by an authorised receptionist (Section 1.2); the method and the receptionist are recorded.	Database: `ck_visits_verified_by`, `ck_visits_walk_in_verified`
Arrival outside visiting hours	Check-in is compared with `operating_hours` when `enforce_operating_hours` is on; the refusal is logged in `pass_scans`. An exception is an `exception_granted` event.	Hour check in the application; database requires actor and reason
Pass expires while the visitor is inside	An expired pass still checks the visitor out. An extension is requested and decided in `visit_extensions`, which moves the visit end and the pass expiry.	Database records who asked and who decided; who may do so is an open decision
Visitor never checks out	Overdue reminder (`overdue_notified_at`), shown as unconfirmed on the on-site list, then `auto-closed` at closing time with no check-out time.	Database keeps auto-closed apart from checked-out; job in the application
Possible blacklist match	Visit held in `security-review`; the match is reviewed and a decision recorded (Section 4.4). Reception sees a neutral message.	Database refuses to schedule or admit until security approves
Lost or damaged QR or badge	Two options are proposed for decision: staff verify and handle movements by hand, or the existing pass gets a new token (Section 4.3).	Database requires a staff member for manual movements and reissues
Kiosk unavailable	Staff check visitors in and out from the dashboard.	Database: each entry and exit names a device or a staff member
Failure or retry during an operation	Each operation runs in one transaction; `idempotency_keys` replays the stored result of a retried request, and `version` rejects conflicting updates.	Database keys and transactions; replay in the application

1.4 Main table inventory

TABLE	AREA	PURPOSE	SECTION
`sites`	SITES	Lists the organisation's sites, such as offices or buildings, where visits can take place.	§2.1
`site_settings`	SITES	Holds the visitor rules for each site, such as opening hours, the default visit length and how long a walk-in request may wait for a decision.	§2.2
`consent_notices`	SITES	Stores each version of the privacy notice and NDA text that visitors are asked to accept, either for every site or for one site.	§2.3
`departments`	SITES	Lists the departments or teams that hosts belong to.	§2.4
`gates`	SITES	Lists the named entrances at each site, such as a main lobby or a loading bay.	§2.5

TABLE	AREA	PURPOSE	SECTION
devices	SITES	Records each kiosk or scanner registered at a site, together with the secure codes used to pair it.	§2.6
people	IDENTITY	Stores the basic details of each person known to the VMS, such as their name, email address and phone number.	§2.7
visitor_profiles	IDENTITY	Keeps visitor-specific information about a person, such as how often they have visited and whether they are a VIP or a contractor.	§2.8
person_merge_log	IDENTITY	Records when two records for the same person are combined into one, who did it and why.	§2.9
users	AUTH	Stores the login account for each staff member who uses the dashboard.	§2.10
user_credentials	AUTH	Stores the securely hashed password for each login account.	§2.11
roles	AUTH	Lists the available staff roles, such as receptionist or site administrator.	§2.12
permissions	AUTH	Lists the individual actions the system can allow or block, such as approving a visit.	§2.13
role_permissions	AUTH	Links each role to the permissions it grants.	§2.14
user_roles	AUTH	Records which roles each staff member holds and at which sites.	§2.15
user_sessions	AUTH	Keeps a record of each signed-in session and a fingerprint of its refresh token.	§2.16
password_reset_requests	AUTH	Tracks the password reset links sent to staff, including when they expire and whether they have been used.	§2.17
authentication_events	AUTH	Keeps a running log of sign-ins, failed attempts, lockouts and password resets.	§2.18
hosts	VISITS	Lists the employees that visitors can come to see, with their employee ID, department and home site.	§2.19
visits	VISITS	Stores each visit from the moment it is requested until it is closed: who is visiting whom, at which site, when, how the visitor was verified and the current status.	§2.20
visit_events	VISITS	Keeps a step-by-step history of each visit, such as approval, check-in, stepping out, extensions, exceptions and check-out, with who did it and why.	§2.21
visit_extensions	VISITS	Records each request to extend a visit beyond its planned end, and the decision on it: who asked, who decided, the previous and new end times and the reason.	§2.22

TABLE	AREA	PURPOSE	SECTION
verification_codes	VISITS	Holds the short-lived one-time codes used to identify a walk-in visitor when they first register, either sent by text message or email or issued by reception after an ID check.	§2.23
passes	VISITS	Stores the QR pass for a visit and how long it is valid.	§2.24
pass_scans	VISITS	Records every time a pass is scanned or checked, and whether it was accepted or why it was refused.	§2.25
idempotency_keys	VISITS	Remembers recent requests so that a request sent twice, for example after a network drop at a kiosk, has its effect only once.	§2.26
blacklist_entries	SECURITY	Lists people who should be refused entry or need security approval, with the reason, the person responsible and how long the restriction lasts.	§2.27
blacklist_matches	SECURITY	Records each possible match between a visitor and a restricted-entry record, and the outcome of the human review of that match.	§2.28
security_approvals	SECURITY	Records the security decision on whether a visitor held for review may proceed, and the reason for it.	§2.29
audit_logs	AUDIT	Keeps a lasting record of important actions, such as who approved a visit or changed an account.	§2.30
notifications	AUDIT	Stores messages for staff, such as "your visitor has arrived", and tracks delivery of any email or text copy.	§2.31
retention_policies	OPS	Sets how long each kind of record is kept and whether it is deleted or anonymised afterwards.	§2.32
retention_holds	OPS	Marks a visit or a person whose records must be kept beyond the normal retention period, for example during a legal case or an investigation.	§2.33
job_runs	OPS	Records each run of a background task, such as the nightly clean-up, a database backup or a test restore, and whether it succeeded.	§2.34

1.5 All foreign keys

Every proposed foreign key, with its cardinality (derived from primary keys and unique constraints) and intended delete behaviour. Many-to-many relationships use the junction tables `role_permissions` (roles ↔ permissions) and `user_roles` (users ↔ roles, per site).

TABLE	COLUMN(S)	REFERENCES	CARDINALITY	ON DELETE
site_settings	site_id	sites	one-to-one	cascade
consent_notices	site_id	sites	many-to-one · optional	restrict
consent_notices	created_by	users	many-to-one	restrict

TABLE	COLUMN(S)	REFERENCES	CARDINALITY	ON DELETE
gates	site_id	sites	many-to-one	restrict
devices	enrolled_by	users	many-to-one · optional	restrict
devices	gate_id	gates	many-to-one · optional	restrict
devices	site_id	sites	many-to-one	restrict
people	created_by	users	many-to-one · optional	set null
people	merged_into_id	people	many-to-one · optional	restrict
people	updated_by	users	many-to-one · optional	set null
visitor_profiles	person_id	people	one-to-one	restrict
person_merge_log	merged_by	users	many-to-one	restrict
person_merge_log	source_person_id	people	many-to-one	restrict
person_merge_log	target_person_id	people	many-to-one	restrict
users	person_id	people	one-to-one	restrict
user_credentials	user_id	users	one-to-one	cascade
role_permissions	permission_id	permissions	many-to-one	cascade
role_permissions	role_id	roles	many-to-one	cascade
user_roles	site_id	sites	many-to-one · optional	restrict
user_roles	user_id	users	many-to-one	cascade
user_roles	granted_by	users	many-to-one · optional	set null
user_roles	role_id	roles	many-to-one	restrict
user_sessions	user_id	users	many-to-one	cascade
password_reset_requests	initiated_by	users	many-to-one · optional	set null
password_reset_requests	user_id	users	many-to-one; unique where (consumed_at IS NULL) AND (cancelled_at IS NULL)	cascade
authentication_events	user_id	users	many-to-one · optional	set null
hosts	department_id	departments	many-to-one	restrict
hosts	person_id	people	one-to-one	restrict
hosts	site_id	sites	many-to-one	restrict
hosts	user_id	users	one-to-one · optional	restrict
visits	created_by_device_id	devices	many-to-one · optional	restrict
visits	host_id	hosts	many-to-one	restrict
visits	site_id	sites	many-to-one	restrict

TABLE	COLUMN(S)	REFERENCES	CARDINALITY	ON DELETE
visits	visitor_person_id	people	many-to-one	restrict
visits	created_by_id	users	many-to-one · optional	restrict
visits	submitted_by_person_id	people	many-to-one · optional	restrict
visits	verified_by	users	many-to-one · optional	restrict
visits	privacy_notice_id	consent_notices	many-to-one · optional	restrict
visits	nda_notice_id	consent_notices	many-to-one · optional	restrict
visit_events	pass_id	passes	many-to-one · optional	restrict
visit_events	visit_id	visits	many-to-one	restrict
visit_events	actor_user_id	users	many-to-one · optional	restrict
visit_events	device_id	devices	many-to-one · optional	restrict
visit_events	gate_id	gates	many-to-one · optional	restrict
visit_extensions	visit_id	visits	many-to-one; unique where status = 'pending'	restrict
visit_extensions	requested_by	users	many-to-one	restrict
visit_extensions	decided_by	users	many-to-one · optional	restrict
verification_codes	visit_id	visits	many-to-one; unique where consumed_at IS NULL AND cancelled_at IS NULL	cascade
verification_codes	issued_by	users	many-to-one · optional	restrict
passes	visit_id	visits	one-to-one	restrict
passes	issued_by_device_id	devices	many-to-one · optional	restrict
passes	issued_by_user_id	users	many-to-one · optional	restrict
passes	revoked_by	users	many-to-one · optional	restrict
pass_scans	actor_user_id	users	many-to-one · optional	restrict
pass_scans	device_id	devices	many-to-one · optional	restrict
pass_scans	pass_id	passes	many-to-one · optional	restrict
pass_scans	site_id	sites	many-to-one · optional	restrict
blacklist_entries	created_by	users	many-to-one	restrict
blacklist_entries	lifted_by	users	many-to-one · optional	restrict
blacklist_entries	person_id	people	many-to-one · optional	restrict
blacklist_entries	site_id	sites	many-to-one · optional	restrict
blacklist_matches	blacklist_entry_id	blacklist_entries	many-to-one	restrict
blacklist_matches	reviewed_by	users	many-to-one · optional	restrict

TABLE	COLUMN(S)	REFERENCES	CARDINALITY	ON DELETE
blacklist_matches	visit_id	visits	many-to-one	restrict
security_approvals	visit_id	visits	many-to-one; unique where status = 'pending'	restrict
security_approvals	decided_by	users	many-to-one · optional	restrict
audit_logs	actor_device_id	devices	many-to-one · optional	restrict
audit_logs	actor_user_id	users	many-to-one · optional	restrict
notifications	recipient_id	users	many-to-one	restrict
notifications	visit_id	visits	many-to-one · optional	set null
retention_policies	site_id	sites	many-to-one · optional	restrict
retention_policies	approved_by	users	many-to-one · optional	restrict
retention_holds	placed_by	users	many-to-one	restrict
retention_holds	released_by	users	many-to-one · optional	restrict
retention_holds	visit_id	visits	many-to-one · optional	restrict
retention_holds	person_id	people	many-to-one · optional	restrict

2 Database tables

One section per table, in the order of the inventory. Each lists the proposed columns, data types, keys and defaults, followed by the proposed constraints, indexes and triggers.

PK Primary key FK → table Foreign key and the table it references UUID Unique identifier TIMESTAMPTZ Date and time with timezone

2.1 sites

SITES

PURPOSE Lists the organisation's sites, such as offices or buildings, where visits can take place. Visits, hosts, kiosks and staff access are all tied to a site.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required	Unique identifier.
name	VARCHAR(180)	Unique Required	Display name of the site.
address	VARCHAR(300)	Optional	Physical address.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
status	TEXT	Required · default `'active'`	active, inactive or archived.

is_active	BOOLEAN	Generated: status = 'active'	True when status is active (generated column).
code	VARCHAR(40)	Optional	Optional short site code.
timezone	VARCHAR(64)	Required · default `'UTC'`	IANA timezone name; opening hours are read in this zone.
archived_at	TIMESTAMPTZ	Optional	When the site was archived.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.

CONSTRAINT	DEFINITION	NAME
Unique index	`lower((code)) WHERE code IS NOT NULL`	`uq_sites_code`
Check	`(status = 'archived') = (archived_at IS NOT NULL)`	`ck_sites_archived_at`
Check	`status IN ('active', 'inactive', 'archived')`	`ck_sites_status`
Trigger	`set_updated_at()`	`trg_sites_updated_at`

2.2 site_settings

SITES

PURPOSE Holds the visitor rules for each site, such as opening hours, the default visit length and how long a walk-in request may wait for a decision. This lets each site work a little differently without code changes.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
site_id	UUID	PK FK → sites *on delete cascade* Required	The site these settings belong to.
operating_hours	JSONB	Required · default `'{}'`	Visiting hours per weekday (JSON).
enforce_operating_hours	BOOLEAN	Required · default `false`	Whether check-in outside visiting hours is refused unless an exception is granted.
default_visit_hours	INTEGER	Required · default `8`	Default visit length for walk-ins.
early_checkin_minutes	INTEGER	Required · default `30`	How long before the visit start a visitor may check in.
walk_in_requires_host_approval	BOOLEAN	Required · default `true`	Whether walk-ins wait for the host's decision; if not, an authorised receptionist decides. Never automatic.
pending_escalation_minutes	INTEGER	Required · default `10`	Minutes a request may wait before reception is alerted.

pending_expiry_minutes	INTEGER	Required · default `60`	Minutes after which an undecided request expires.
overdue_grace_minutes	INTEGER	Required · default `30`	Minutes after the visit end before a visitor is shown as overdue.
allow_reentry	BOOLEAN	Required · default `true`	Whether visitors may sign out and return on the same pass.
require_nda	BOOLEAN	Required · default `true`	Whether visitors must accept the NDA.
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Check	`(default_visit_hours ⩾ 1) AND (default_visit_hours ⩽ 72)`	`ck_site_settings_default_visit_hours`
Check	`(overdue_grace_minutes ⩾ 0) AND (overdue_grace_minutes ⩽ 1440)`	`ck_site_settings_overdue_grace`
Check	`early_checkin_minutes BETWEEN 0 AND 240`	`ck_site_settings_early_checkin`
Check	`pending_escalation_minutes ⩾ 1 AND pending_expiry_minutes > pending_escalation_minutes AND pending_expiry_minutes ⩽ 1440`	`ck_site_settings_pending_windows`
Trigger	`set_updated_at()`	`trg_site_settings_updated_at`

Application rule. The application is intended to create the settings row together with each site. Opening hours are checked in the site's local time, and the escalation and expiry windows are applied by a background job.

2.3 consent_notices

SITES

PURPOSE Stores each version of the privacy notice and NDA text that visitors are asked to accept, either for every site or for one site. Visits point to the exact version a visitor accepted, so it is always possible to show what they agreed to.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
site_id	UUID	FK → sites *on delete restrict* Optional	Site the notice applies to; empty means every site.
kind	TEXT	Required	privacy or nda.
notice_version	INTEGER	Required	Version number, increasing per site and kind.
body	TEXT	Required	Full text shown to the visitor.
effective_from	TIMESTAMPTZ	Required · default `now()`	When this version starts being shown.
created_by	UUID	FK → users *on delete restrict* Required	Account that published it.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.

CONSTRAINT	DEFINITION	NAME
Unique index	`site_id, kind, notice_version (nulls not distinct)`	`uq_consent_notices_version`
Check	`kind IN ('privacy', 'nda')`	`ck_consent_notices_kind`
Check	`notice_version ⩾ 1`	`ck_consent_notices_version`
Trigger	`reject_history_change()`	`trg_consent_notices_append_only`

Application rule. A new version is added whenever the text changes; existing versions are never edited (the append-only trigger enforces this). The kiosk shows the newest version in effect for the site.

2.4 departments

SITES

PURPOSE Lists the departments or teams that hosts belong to. Visitors and reception can then find a host by department.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
name	VARCHAR(120)	Required	Department name.
status	TEXT	Required · default `'active'`	active or inactive.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Unique index	`lower((name))`	`uq_departments_name`
Check	`length(TRIM(BOTH FROM name)) > 0`	`ck_departments_name_not_blank`
Check	`status IN ('active', 'inactive')`	`ck_departments_status`
Trigger	`set_updated_at()`	`trg_departments_updated_at`

2.5 gates

SITES

PURPOSE Lists the named entrances at each site, such as a main lobby or a loading bay. Kiosks and check-in records can then be linked to the entrance used; a site with a single entrance needs none.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
site_id	UUID	FK → sites *on delete restrict* Required	Site the row belongs to.
name	VARCHAR(120)	Required	Gate name.
status	TEXT	Required · default `'active'`	active or inactive.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Unique	`site_id, name`	`uq_gates_site_name`
Check	`status IN ('active', 'inactive')`	`ck_gates_status`

CONSTRAINT	DEFINITION	NAME
Trigger	`set_updated_at()`	`trg_gates_updated_at`

2.6 devices

SITES

PURPOSE Records each kiosk or scanner registered at a site, together with the secure codes used to pair it. Only registered, active devices are meant to be able to check visitors in and out, and only at their own site.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
site_id	UUID	FK → sites *on delete restrict* Required	Site the row belongs to.
gate_id	UUID	FK → gates *on delete restrict* Optional	Gate the device is mounted at, if any.
name	VARCHAR(120)	Required	Device name, unique per site.
device_type	TEXT	Required	kiosk or scanner.
credential_hash	VARCHAR(64)	Unique Unique where credential_hash IS NOT NULL Optional	Keyed hash of the device key (set when paired).
status	TEXT	Required · default `'pending'`	pending, active, disabled or retired.
enrolled_at	TIMESTAMPTZ	Optional	When the device was paired.
enrolled_by	UUID	FK → users *on delete restrict* Optional	Account that registered the device.
last_seen_at	TIMESTAMPTZ	Optional	Last authenticated request from the device.
app_version	VARCHAR(40)	Optional	Kiosk app version last reported.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.
pairing_code_hash	VARCHAR(64)	Unique Unique where pairing_code_hash IS NOT NULL Optional	Keyed hash of the one-time pairing code.
pairing_expires_at	TIMESTAMPTZ	Optional	When the pairing code expires.

CONSTRAINT	DEFINITION	NAME
Unique	site_id, name	uq_devices_site_name
Unique index	credential_hash WHERE credential_hash IS NOT NULL	uq_devices_credential_hash
Unique index	pairing_code_hash WHERE pairing_code_hash IS NOT NULL	uq_devices_pairing_code_hash
Check	(status <> 'active') OR ((credential_hash IS NOT NULL) AND (enrolled_at IS NOT NULL))	ck_devices_active_enrolled
Check	status IN ('pending', 'active', 'disabled', 'retired')	ck_devices_status
Check	device_type IN ('kiosk', 'scanner')	ck_devices_type
Index	btree (site_id) WHERE status = 'active'	ix_devices_site_active
Trigger	set_updated_at()	trg_devices_updated_at

Application rule. Pairing codes are intended to expire after 15 minutes and work only once; device keys are hashed with a server-side key. A device acts only for its own site; scans for visits at another site are refused.

2.7 people

IDENTITY

PURPOSE Stores the basic details of each person known to the VMS, such as their name, email address and phone number. Visitors, hosts and staff all refer to this table instead of repeating the same person's details.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
full_name	VARCHAR(160)	Required	Person's name.
email	VARCHAR(320)	Optional	Contact email (stored lowercase; not unique).
phone	VARCHAR(40)	Optional	Contact number (not unique).
company	VARCHAR(160)	Optional	Company the visitor represents.
identity_status	TEXT	Required · default `'active'`	active, merged or anonymized.
merged_into_id	UUID	FK → people *on delete restrict* Optional	Surviving record when this one was merged.
anonymized_at	TIMESTAMPTZ	Optional	When personal details were removed.
created_by	UUID	FK → users *on delete set null* Optional	Account that created the record.

updated_by	UUID	FK → users *on delete set null* Optional	Account that last changed the record.
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Check	(identity_status = 'anonymized') = (anonymized_at IS NOT NULL)	ck_people_anonymized_at
Check	(email IS NULL) OR (email = lower((email)))	ck_people_email_lowercase
Check	length(TRIM(BOTH FROM full_name)) > 0	ck_people_full_name_not_blank
Check	identity_status IN ('active', 'merged', 'anonymized')	ck_people_identity_status
Check	(identity_status = 'merged') = (merged_into_id IS NOT NULL)	ck_people_merged_into
Index	gin (full_name (trigram))	ix_people_full_name_trgm
Index	btree (lower((email))) WHERE email IS NOT NULL	ix_people_email
Index	btree (phone) WHERE phone IS NOT NULL	ix_people_phone
Index	btree ("right"(regexp_replace((phone), '[^0-9]', '', 'g'), 10)) WHERE phone IS NOT NULL	ix_people_phone_key
Trigger	set_updated_at()	trg_people_updated_at

Application rule. Matching a returning visitor (same email or phone and a similar name) and merging duplicates are intended to be handled by the application, not by database rules. A name alone never links two people.

2.8 visitor_profiles

IDENTITY

PURPOSE Keeps visitor-specific information about a person, such as how often they have visited and whether they are a VIP or a contractor. This helps staff recognise returning visitors; it does not decide whether someone may enter.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
person_id	UUID	PK FK → people *on delete restrict* Required	Person this profile belongs to.
classification	TEXT	Required · default `'standard'`	standard, frequent, contractor or vip.
identity_review	BOOLEAN	Required · default `false`	Flagged as a possible duplicate for staff review.
first_visit_at	TIMESTAMPTZ	Optional	First check-in.
last_visit_at	TIMESTAMPTZ	Optional	Most recent check-in.
visit_count	INTEGER	Required · default `0`	Number of visits with a check-in.
notes	VARCHAR(1000)	Optional	Administrator notes.
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Check	`visit_count ⩾ 0`	`ck_visitor_profiles_visit_count`
Check	`classification IN ('standard', 'frequent', 'contractor', 'vip')`	`ck_visitor_profiles_classification`
Index	`btree (classification)`	`ix_visitor_profiles_classification`
Index	`btree (last_visit_at DESC NULLS LAST)`	`ix_visitor_profiles_last_visit`
Trigger	`set_updated_at()`	`trg_visitor_profiles_updated_at`

Application rule. Visit counts are intended to be updated by the application at check-in. Restricting entry is done only through blacklist entries, never through this table.

2.9 person_merge_log

IDENTITY

PURPOSE Records when two records for the same person are combined into one, who did it and why. This keeps a trail of every merge of duplicate records.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
source_person_id	UUID	FK → people *on delete restrict* Required	Duplicate record that was merged away.
target_person_id	UUID	FK → people *on delete restrict* Required	Record that was kept.
merged_by	UUID	FK → users *on delete restrict* Required	Administrator who merged.
reason	VARCHAR(500)	Required	Reason given.
moved_counts	JSONB	Required	How many visits and other references moved (JSON).
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.

CONSTRAINT	DEFINITION	NAME
Check	`source_person_id <> target_person_id`	`ck_person_merge_distinct`

2.10 users

AUTH

PURPOSE Stores the login account for each staff member who uses the dashboard. Login details are kept apart from personal details because not every person needs an account.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required	Unique identifier.
email	VARCHAR(320)	Unique Required	Login email (lowercase).
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
person_id	UUID	FK → people *on delete restrict* Unique Required	Person associated with the account.
account_status	TEXT	Required · default `'active'`	invited, active or disabled.
last_login_at	TIMESTAMPTZ	Optional	Last successful sign-in.

failed_login_count	INTEGER	Required · default `0`	Consecutive failed sign-ins.
locked_until	TIMESTAMPTZ	Optional	End of a temporary lockout, if any.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.
is_active	BOOLEAN	Generated: account_status = 'active'	True when account_status is active (generated column).

CONSTRAINT	DEFINITION	NAME
Unique index	`email`	`ix_users_email`
Check	`failed_login_count ⩾ 0`	`ck_users_failed_login_count`
Check	`account_status IN ('invited', 'active', 'disabled')`	`ck_users_account_status`
Check	`email = lower(email)`	`ck_users_email_lowercase`
Trigger	`set_updated_at()`	`trg_users_updated_at`

> **Application rule.** Disabling an account is intended to revoke its sessions at the same time. Temporary lockout after repeated failed sign-ins uses locked_until.

2.11 user_credentials

AUTH

PURPOSE Stores the securely hashed password for each login account. Keeping it in its own table limits where password data can appear.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
user_id	UUID	PK FK → users *on delete cascade* Required	Account the credential belongs to.
password_hash	TEXT	Required	Argon2id hash, never the password.
password_changed_at	TIMESTAMPTZ	Required · default `now()`	When the password was last set.
must_change	BOOLEAN	Required · default `false`	Whether a change is required at next sign-in.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Check	`password_hash ~~ '$argon2id$%'`	`ck_user_credentials_argon2id`

CONSTRAINT	DEFINITION	NAME
Trigger	set_updated_at()	trg_user_credentials_updated_at

Application rule. Hashing with Argon2id is done by the application; the database only checks that the stored value is an Argon2id hash.

2.12 roles

AUTH

PURPOSE Lists the available staff roles, such as receptionist or site administrator. A staff member may hold several roles, and together they decide what the person is allowed to do.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	SMALLINT	PK Required · default nextval('roles_id_seq'::regclass)	Role identifier.
code	VARCHAR(40)	Unique Required	super-admin, site-admin, receptionist or host.
scope	TEXT	Required	global, global_or_site or site.

CONSTRAINT	DEFINITION	NAME
Check	scope IN ('global', 'global_or_site', 'site')	ck_roles_scope

2.13 permissions

AUTH

PURPOSE Lists the individual actions the system can allow or block, such as approving a visit. The API checks these before carrying out an action.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	SMALLINT	PK Required · default nextval('permissions_id_seq'::regclass)	Permission identifier.
code	VARCHAR(60)	Unique Required	Permission code, e.g. visit.approve.

2.14 role_permissions

AUTH

PURPOSE Links each role to the permissions it grants. This makes it possible to change what a role can do without changing code.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
role_id	SMALLINT	PK *composite* FK → roles *on delete cascade* Required	Role.
permission_id	SMALLINT	PK *composite* FK → permissions *on delete cascade* Required	Permission granted to the role.

2.15 user_roles

AUTH

PURPOSE Records which roles each staff member holds and at which sites. This is how the system knows that a receptionist works at one site but not another.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
user_id	UUID	FK → users *on delete cascade* Required	Account receiving the role.
role_id	SMALLINT	FK → roles *on delete restrict* Required	Role assigned.
site_id	UUID	FK → sites *on delete restrict* Optional	Site the role applies to; empty means all sites.
granted_by	UUID	FK → users *on delete set null* Optional	Account that granted the role.
granted_at	TIMESTAMPTZ	Required · default `now()`	When it was granted.

CONSTRAINT	DEFINITION	NAME
Unique index	`user_id, role_id, COALESCE(site_id, '00000000-0000-0000-0000-000000000000')`	`uq_user_roles_assignment`
Index	`btree (site_id)`	`ix_user_roles_site`
Trigger	`enforce_user_role_scope()`	`trg_user_roles_scope`

Application rule. Only the roles that cover the site of the record being used count, so a host role at one site and a receptionist role at another never combine into receptionist rights at the first site. Record-level rules then apply on top (Section 6). Removing a role takes effect at the next token refresh at the latest, and the change is recorded in audit_logs.

2.16 user_sessions

AUTH

PURPOSE Keeps a record of each signed-in session and a fingerprint of its refresh token. This allows a sign-in to be ended straight away, for example on logout, after a password change or when access is withdrawn.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
user_id	UUID	FK → users *on delete cascade* Required	Account signed in.
refresh_token_hash	VARCHAR(64)	Unique Required	SHA-256 of the current refresh token.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
expires_at	TIMESTAMPTZ	Required	When the session ends.
last_used_at	TIMESTAMPTZ	Optional	Last token refresh.
revoked_at	TIMESTAMPTZ	Optional	When it was revoked (logout, password change, deactivation).
revoke_reason	VARCHAR(40)	Optional	Why it was revoked.
user_agent	VARCHAR(300)	Optional	Browser user agent.
ip	INET	Optional	Client IP address.

CONSTRAINT	DEFINITION	NAME
Check	`expires_at > created_at`	`ck_user_sessions_expiry`
Check	`(revoked_at IS NULL) = (revoke_reason IS NULL)`	`ck_user_sessions_revoke_reason`
Index	`btree (user_id) WHERE revoked_at IS NULL`	`ix_user_sessions_user_active`

Application rule. Refresh tokens are random, stored only as a hash and replaced on every refresh. The API is intended to reject any request whose session has been revoked or has expired.

2.17 password_reset_requests

AUTH

PURPOSE Tracks the password reset links sent to staff, including when they expire and whether they have been used. This makes sure each link works only once and only for a short time.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
user_id	UUID	FK → users *on delete cascade* Unique where (consumed_at IS NULL) AND (cancelled_at IS NULL) Required	Account being reset.
token_hash	VARCHAR(64)	Unique Required	Keyed hash of the reset token.
initiated_by	UUID	FK → users *on delete set null* Optional	Administrator who started it, if not self-service.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
expires_at	TIMESTAMPTZ	Required	When the link expires.
consumed_at	TIMESTAMPTZ	Optional	When the link was used.
cancelled_at	TIMESTAMPTZ	Optional	When a newer request cancelled it.
request_ip	INET	Optional	IP address of the request.

CONSTRAINT	DEFINITION	NAME
Unique index	`user_id WHERE (consumed_at IS NULL) AND (cancelled_at IS NULL)`	`uq_password_reset_live`
Check	`expires_at > created_at`	`ck_password_reset_expiry`
Check	`NOT ((consumed_at IS NOT NULL) AND (cancelled_at IS NOT NULL))`	`ck_password_reset_single_outcome`
Index	`btree (user_id, created_at DESC)`	`ix_password_reset_user_created`

Application rule. Reset links are sent only to the account's login email. Expiry, single use (through a conditional update) and rate limits are intended to be enforced by the application.

2.18 authentication_events

AUTH

PURPOSE Keeps a running log of sign-ins, failed attempts, lockouts and password resets. This helps to detect misuse and investigate account problems.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	BIGINT	PK Identity (always)	Unique identifier.
user_id	UUID	FK → users *on delete set null* Optional	Account involved, if known.
event_type	VARCHAR(40)	Required	e.g. login.success, login.failure, reset.completed.
ip	INET	Optional	Client IP address.
user_agent	VARCHAR(300)	Optional	Browser user agent.
metadata	JSONB	Required · default `'{}'`	Structured, non-sensitive details (JSON).
occurred_at	TIMESTAMPTZ	Required · default `now()`	When it happened.

CONSTRAINT	DEFINITION	NAME
Index	btree (ip, event_type, occurred_at DESC)	ix_authentication_events_ip_type
Index	btree (occurred_at)	ix_authentication_events_occurred
Index	btree (user_id, occurred_at DESC)	ix_authentication_events_user
Trigger	reject_history_change()	trg_authentication_events_append_only

2.19 hosts

VISITS

PURPOSE Lists the employees that visitors can come to see, with their employee ID, department and home site. Each host links to the person's details and, if they have one, their login account.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required	Unique identifier.
site_id	UUID	FK → sites *on delete restrict* Required	Host's home site.
employee_id	VARCHAR(80)	Required	Internal employee identifier (unique, any letter case).
person_id	UUID	FK → people *on delete restrict* Unique Required	Person who acts as the host.
user_id	UUID	FK → users *on delete restrict* Unique Unique where user_id IS NOT NULL Optional	Login account of the host, if any.
department_id	UUID	FK → departments *on delete restrict* Required	Department.
status	TEXT	Required · default `'active'`	active or inactive.
deactivated_at	TIMESTAMPTZ	Optional	When the host was deactivated.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.
active	BOOLEAN	Generated: status = 'active'	True when status is active (generated column).

CONSTRAINT	DEFINITION	NAME
Unique index	`user_id WHERE user_id IS NOT NULL`	`uq_hosts_user_id`
Unique index	`lower((employee_id))`	`uq_hosts_employee_id`
Check	`(status = 'inactive') = (deactivated_at IS NOT NULL)`	`ck_hosts_deactivated_at`
Check	`status IN ('active', 'inactive')`	`ck_hosts_status`

CONSTRAINT	DEFINITION	NAME
Index	gin (employee_id (trigram))	ix_hosts_employee_id_trgm
Index	btree (site_id, status)	ix_hosts_site_status
Trigger	set_updated_at()	trg_hosts_updated_at

Application rule. Deactivating a host is intended to flag their upcoming visits for reassignment; the database rejects new visits for an inactive host.

2.20 visits

VISITS

PURPOSE Stores each visit from the moment it is requested until it is closed: who is visiting whom, at which site, when, how the visitor was verified and the current status. Pre-registered and walk-in visits use the same record, so each occasion has exactly one place to look.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required	Unique identifier.
site_id	UUID	FK → sites *on delete restrict* Required	Site being visited.
visitor_person_id	UUID	FK → people *on delete restrict* Required	Visitor for this visit.
host_id	UUID	FK → hosts *on delete restrict* Required	Host being visited.
submitted_by_person_id	UUID	FK → people *on delete restrict* Optional	Person who registered on the visitor's behalf, e.g. an assistant.
created_by_id	UUID	FK → users *on delete restrict* Optional	Staff account that created it.
created_by_device_id	UUID	FK → devices *on delete restrict* Optional	Kiosk that created it, if any.
source	VARCHAR(24)	Required · default `'pre-registered'`	pre-registered or walk-in.
category	TEXT	Required · default `'business'`	business, interview, contractor, delivery, personal or other.
priority	BOOLEAN	Required · default `false`	Flagged for priority handling (does not bypass approval).
purpose	VARCHAR(500)	Required	Reason for the visit.

starts_at	TIMESTAMPTZ	Required	Expected start.
ends_at	TIMESTAMPTZ	Required	Current expected end, after any approved extension; may be days later for longer work.
status	TEXT	Required	See Section 1.1.
verification_method	TEXT	Optional	How a walk-in was identified at registration: otp_sms, otp_email, reception_id_check or reception_code.
verified_at	TIMESTAMPTZ	Optional	When that identification was completed (once per visit).
verified_by	UUID	FK → users *on delete restrict* Optional	Receptionist who verified, for the reception methods.
checked_in_at	TIMESTAMPTZ	Optional	First entry.
checked_out_at	TIMESTAMPTZ	Optional	Confirmed final departure.
overdue_notified_at	TIMESTAMPTZ	Optional	When the overdue reminder was sent.
closed_at	TIMESTAMPTZ	Optional	When the visit reached a closed status; starts the retention clock.
privacy_notice_id	UUID	FK → consent_notices *on delete restrict* Optional	Privacy notice version accepted.
privacy_consent_at	TIMESTAMPTZ	Optional	When the privacy notice was accepted.
nda_notice_id	UUID	FK → consent_notices *on delete restrict* Optional	NDA version accepted.
nda_signed_at	TIMESTAMPTZ	Optional	When the NDA was accepted.
anonymized_at	TIMESTAMPTZ	Optional	When retention removed personal details.
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Check	`category IN ('business', 'interview', 'contractor', 'delivery', 'personal', 'other')`	`ck_visits_category`

CONSTRAINT	DEFINITION	NAME
Check	ends_at > starts_at	ck_visits_period
Check	(closed_at IS NOT NULL) = (status IN ('checked-out', 'auto-closed', 'no-show', 'expired', 'cancelled', 'rejected'))	ck_visits_closed_at
Check	status IN ('pending', 'security-review', 'scheduled', 'checked-in', 'temporarily-out', 'checked-out', 'auto-closed', 'no-show', 'expired', 'cancelled', 'rejected')	ck_visits_status
Check	status NOT IN ('checked-in', 'temporarily-out', 'checked-out', 'auto-closed') OR checked_in_at IS NOT NULL	ck_visits_checked_in
Check	(status = 'checked-out') = (checked_out_at IS NOT NULL)	ck_visits_checked_out
Check	checked_out_at IS NULL OR checked_out_at ⩾ checked_in_at	ck_visits_checkout_order
Check	source IN ('pre-registered', 'walk-in')	ck_visits_source
Check	submitted_by_person_id IS NULL OR submitted_by_person_id <> visitor_person_id	ck_visits_submitter
Check	verification_method IN ('otp_sms', 'otp_email', 'reception_id_check', 'reception_code')	ck_visits_verification_method
Check	(verification_method IS NULL) = (verified_at IS NULL)	ck_visits_verified_at
Check	verification_method NOT IN ('reception_id_check', 'reception_code') OR verified_by IS NOT NULL	ck_visits_verified_by
Check	(privacy_consent_at IS NULL) = (privacy_notice_id IS NULL)	ck_visits_privacy_notice
Check	(nda_signed_at IS NULL) = (nda_notice_id IS NULL)	ck_visits_nda_notice
Check	source <> 'walk-in' OR status IN ('pending', 'security-review', 'expired', 'cancelled', 'rejected') OR verification_method IS NOT NULL	ck_visits_walk_in_verified
Index	btree (host_id, starts_at DESC)	ix_visits_host_start
Index	btree (site_id, status, starts_at)	ix_visits_site_status_start
Index	btree (visitor_person_id, starts_at DESC)	ix_visits_visitor_start
Index	btree (closed_at) WHERE anonymized_at IS NULL AND closed_at IS NOT NULL	ix_visits_retention_pending
Index	btree (created_at) WHERE status IN ('pending', 'security-review')	ix_visits_awaiting_decision
Index	btree (site_id, updated_at)	ix_visits_site_updated
Index	btree (site_id) WHERE status = 'checked-in'	ix_visits_on_site
Index	btree (ends_at) WHERE status IN ('checked-in', 'temporarily-out')	ix_visits_open_by_end
Trigger	reject_inactive_host()	trg_visits_active_host
Trigger	set_updated_at()	trg_visits_updated_at

CONSTRAINT	DEFINITION	NAME
Trigger	`reject_uncleared_visit()`	`trg_visits_security_clearance`

Application rule. Which status changes are allowed (Section 1.1) is intended to be enforced by the application, using version to reject conflicting updates; the database restricts the values and ties check-in, check-out and closing times to the statuses that need them. The database also refuses to schedule a walk-in that has no recorded identification, and refuses to schedule or admit a visit whose security review is pending or denied.

2.21 visit_events

VISITS

PURPOSE Keeps a step-by-step history of each visit, such as approval, check-in, stepping out, extensions, exceptions and check-out, with who did it and why. Entries are only ever added, so the full history is preserved.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
visit_id	UUID	FK → visits *on delete restrict* Required	Visit the event belongs to.
event_type	TEXT	Required	e.g. approved, check_in, step_out, extended, exception_granted.
from_status	TEXT	Optional	Status before.
to_status	TEXT	Optional	Status after.
occurred_at	TIMESTAMPTZ	Required · default `now()`	When it happened.
recorded_at	TIMESTAMPTZ	Required · default `now()`	When the server stored it.
actor_user_id	UUID	FK → users *on delete restrict* Optional	Staff account that acted, if any.
device_id	UUID	FK → devices *on delete restrict* Optional	Kiosk that acted, if any.
gate_id	UUID	FK → gates *on delete restrict* Optional	Entrance used, if known.
pass_id	UUID	FK → passes *on delete restrict* Optional	Pass involved, if any.
reason	VARCHAR(500)	Optional	Reason given (required for exceptions).

metadata	JSONB	Required · default `'{}'`	Structured, non-sensitive details (JSON).

CONSTRAINT	DEFINITION	NAME
Check	`event_type IN ('created', 'identity_verified', 'approval_escalated', 'approved', 'rejected', 'expired', 'security_review_requested', 'security_approved', 'security_denied', 'pass_issued', 'pass_reissued', 'pass_revoked', 'check_in', 'step_out', 'reentry', 'check_out', 'extended', 'host_changed', 'exception_granted', 'completed', 'cancelled', 'no_show', 'overdue_escalated', 'auto_closed', 'manual_correction', 'reconciled', 'identity_merged')`	`ck_visit_events_type`
Check	`event_type NOT IN ('approved', 'rejected', 'exception_granted', 'extended', 'host_changed', 'manual_correction', 'pass_reissued') OR actor_user_id IS NOT NULL`	`ck_visit_events_actor`
Check	`event_type NOT IN ('exception_granted', 'host_changed', 'manual_correction', 'pass_reissued') OR reason IS NOT NULL`	`ck_visit_events_reason`
Check	`event_type NOT IN ('check_in', 'step_out', 'reentry', 'check_out') OR device_id IS NOT NULL OR actor_user_id IS NOT NULL`	`ck_visit_events_movement`
Index	`btree (visit_id, occurred_at)`	`ix_visit_events_visit`
Trigger	`reject_history_change()`	`trg_visit_events_append_only`

Application rule. Each status change, exception and pass action is intended to be written in the same transaction as the change itself. An approval by anyone other than the visit's host is intended to carry a reason. occurred_at is when it happened at the kiosk; recorded_at is when the server stored it, which matters for retried requests.

2.22 visit_extensions

VISITS

PURPOSE Records each request to extend a visit beyond its planned end, and the decision on it: who asked, who decided, the previous and new end times and the reason. The planned end before any extension can always be recovered from these rows.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
visit_id	UUID	FK → visits *on delete restrict* Unique where status = 'pending' Required	Visit to be extended.
previous_ends_at	TIMESTAMPTZ	Required	Visit end before this extension.
requested_ends_at	TIMESTAMPTZ	Required	New end asked for.
reason	VARCHAR(500)	Optional	Reason given, where policy requires one.
status	TEXT	Required · default `'pending'`	pending, approved, rejected or withdrawn.
requested_by	UUID	FK → users *on delete restrict* Required	Staff account that asked for it.
requested_at	TIMESTAMPTZ	Required · default `now()`	When it was asked for.
decided_by	UUID	FK → users *on delete restrict* Optional	Staff account that approved or rejected it.
decided_at	TIMESTAMPTZ	Optional	When it was decided.
decision_note	VARCHAR(500)	Optional	Note from the decision maker.

CONSTRAINT	DEFINITION	NAME
Unique index	`visit_id WHERE status = 'pending'`	`uq_visit_extensions_pending`
Check	`status IN ('pending', 'approved', 'rejected', 'withdrawn')`	`ck_visit_extensions_status`
Check	`requested_ends_at > previous_ends_at`	`ck_visit_extensions_later`
Check	`(status IN ('approved', 'rejected')) = (decided_by IS NOT NULL)`	`ck_visit_extensions_decided`
Check	`(decided_by IS NULL) = (decided_at IS NULL)`	`ck_visit_extensions_decided_at`
Index	`btree (visit_id, requested_at)`	`ix_visit_extensions_visit`

Application rule. An approved extension is intended to update visits.ends_at and the pass's expires_at in the same transaction and add an extended event. Who may request and who may approve is a policy decision (Section 7); the database assumes neither that they are the same person nor that they differ.

2.23 verification_codes

VISITS

PURPOSE Holds the short-lived one-time codes used to identify a walk-in visitor when they first register, either sent by text message or email or issued by reception after an ID check. Codes are used only at registration, not at later entries; only a secure fingerprint is kept, and a code stops working after a few wrong attempts.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
visit_id	UUID	FK → visits *on delete cascade* Unique where consumed_at IS NULL AND cancelled_at IS NULL Required	Walk-in visit being identified.
method	TEXT	Required	otp_sms, otp_email or reception_code.
code_hash	VARCHAR(64)	Required	Keyed hash of the code.
issued_by	UUID	FK → users *on delete restrict* Optional	Receptionist who issued a reception code.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
expires_at	TIMESTAMPTZ	Required	When the code stops working.
attempt_count	INTEGER	Required · default `0`	Wrong attempts so far.
max_attempts	INTEGER	Required · default `5`	Attempts allowed.
consumed_at	TIMESTAMPTZ	Optional	When the code was used.
cancelled_at	TIMESTAMPTZ	Optional	When a newer code replaced it.

CONSTRAINT	DEFINITION	NAME
Unique index	`visit_id WHERE consumed_at IS NULL AND cancelled_at IS NULL`	`uq_verification_codes_live`
Check	`method IN ('otp_sms', 'otp_email', 'reception_code')`	`ck_verification_codes_method`
Check	`method <> 'reception_code' OR issued_by IS NOT NULL`	`ck_verification_codes_issuer`
Check	`expires_at > created_at`	`ck_verification_codes_expiry`

CONSTRAINT	DEFINITION	NAME
Check	`attempt_count ⩾ 0 AND attempt_count ⩽ max_attempts`	`ck_verification_codes_attempts`
Check	`NOT (consumed_at IS NOT NULL AND cancelled_at IS NOT NULL)`	`ck_verification_codes_single_outcome`
Index	`btree (expires_at)`	`ix_verification_codes_expires`

Application rule. Codes are intended to be generated, sent and checked by the application during walk-in registration only, hashed with a server-side key, and removed by the clean-up job a day after they expire.

2.24 passes

VISITS

PURPOSE Stores the QR pass for a visit and how long it is valid. Only a secure fingerprint of the current QR code is kept, so the code cannot be recovered from the database and reveals nothing about the visitor.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
visit_id	UUID	FK → visits *on delete restrict* Unique Required	Visit the pass belongs to.
token_hash	VARCHAR(64)	Unique Required	SHA-256 of the current QR token, used for lookup.
status	TEXT	Required · default `'active'`	active, expired or revoked.
valid_from	TIMESTAMPTZ	Required	Start of validity.
expires_at	TIMESTAMPTZ	Required	End of validity for entry.
issued_by_user_id	UUID	FK → users *on delete restrict* Optional	Staff account that issued it.
issued_by_device_id	UUID	FK → devices *on delete restrict* Optional	Kiosk that issued it.
issued_at	TIMESTAMPTZ	Required · default `now()`	When issued.
revoked_at	TIMESTAMPTZ	Optional	When revoked.
revoked_by	UUID	FK → users *on delete restrict* Optional	Account that revoked it.
revoke_reason	VARCHAR(500)	Optional	Reason given, e.g. lost.

created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Check	`(status = 'revoked') = (revoked_at IS NOT NULL)`	`ck_passes_revoked_at`
Check	`expires_at > valid_from`	`ck_passes_validity`
Check	`status IN ('active', 'expired', 'revoked')`	`ck_passes_status`
Index	`btree (expires_at) WHERE status = 'active'`	`ix_passes_expiry_active`
Trigger	`set_updated_at()`	`trg_passes_updated_at`

Application rule. A pass is intended to be issued only while the visit is scheduled, checked in or temporarily out. Each entry checks the pass, the visit status, restrictions and visiting hours; no OTP is asked for. An expired pass still checks a visitor out; a revoked pass does not, and staff record the exit instead. A lost pass is handled by one of the two options in Section 4.3.

2.25 pass_scans

VISITS

PURPOSE Records every time a pass is scanned or checked, and whether it was accepted or why it was refused. This helps to spot misuse and investigate entry problems.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	BIGINT	PK Identity (always)	Sequential identifier.
pass_id	UUID	FK → passes *on delete restrict* Optional	Pass scanned (empty for unknown tokens).
site_id	UUID	FK → sites *on delete restrict* Optional	Site where the scan happened.
device_id	UUID	FK → devices *on delete restrict* Optional	Kiosk that scanned.
actor_user_id	UUID	FK → users *on delete restrict* Optional	Staff account that verified.
action	TEXT	Required	verify, check_in or check_out.
outcome	TEXT	Required	accepted, or the reason it was refused.
scanned_at	TIMESTAMPTZ	Required · default `now()`	When it happened.

CONSTRAINT	DEFINITION	NAME
Check	action IN ('verify', 'check_in', 'check_out')	ck_pass_scans_action
Check	(device_id IS NOT NULL) OR (actor_user_id IS NOT NULL)	ck_pass_scans_actor
Check	outcome IN ('accepted', 'malformed', 'unknown', 'expired', 'not_yet_valid', 'revoked', 'duplicate', 'outside_hours', 'visit_cancelled', 'visit_completed', 'not_approved', 'wrong_site', 'not_eligible', 'security_pending', 'blacklisted')	ck_pass_scans_outcome
Index	btree (pass_id, scanned_at)	ix_pass_scans_pass
Index	btree (site_id, scanned_at) WHERE outcome <> 'accepted'	ix_pass_scans_site_rejected
Trigger	reject_history_change()	trg_pass_scans_append_only

2.26 idempotency_keys

VISITS

PURPOSE Remembers recent requests so that a request sent twice, for example after a network drop at a kiosk, has its effect only once. Entries are kept for about a day.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
scope	VARCHAR(80)	PK *composite* Required	Caller scope (user or device).
key	VARCHAR(120)	PK *composite* Required	Client-supplied key.
request_hash	VARCHAR(64)	Required	Hash of the request body.
response_status	INTEGER	Optional	Stored HTTP status.
response_body	JSONB	Optional	Stored response (JSON).
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
expires_at	TIMESTAMPTZ	Required	When the key may be removed.

CONSTRAINT	DEFINITION	NAME
Index	btree (expires_at)	ix_idempotency_keys_expires

Application rule. Replaying stored responses is handled by the application. A response containing a QR token is never stored here; a retried issue request instead gives the pass a new token, which is safe because the first one was never delivered. Expired keys are intended to be removed by the clean-up job.

2.27 blacklist_entries

SECURITY

PURPOSE Lists people who should be refused entry or need security approval, with the reason, the person responsible and how long the restriction lasts. This is the only place where a person's entry is restricted.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
site_id	UUID	FK → sites *on delete restrict* Optional	Site the restriction applies to; empty means all sites.
person_id	UUID	FK → people *on delete restrict* Optional	Linked person, if known.
match_name	VARCHAR(160)	Optional	Name to match.
match_email	VARCHAR(320)	Optional	Email to match (lowercase).
match_phone_digits	VARCHAR(40)	Optional	Last 10 digits of a phone number.
reason_category	TEXT	Required	safety, theft, harassment, trespass, legal_order, policy_violation or other.
reason_detail	VARCHAR(1000)	Optional	Restricted detail.
severity	TEXT	Required	deny or require_approval, applied once a match is confirmed.
effective_from	TIMESTAMPTZ	Required · default `now()`	Start of the restriction.
effective_until	TIMESTAMPTZ	Optional	End, if any.
review_due_at	TIMESTAMPTZ	Required	Date the record must be reviewed.
status	TEXT	Required · default `'active'`	active, expired or lifted.
created_by	UUID	FK → users *on delete restrict* Required	Account responsible for the entry.
lifted_by	UUID	FK → users *on delete restrict* Optional	Account that lifted it.
lifted_at	TIMESTAMPTZ	Optional	When lifted.
lift_reason	VARCHAR(500)	Optional	Reason given.
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.

updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Check	`(match_email IS NULL) OR (match_email = lower((match_email)))`	`ck_blacklist_email_lowercase`
Check	`(person_id IS NOT NULL) OR (match_email IS NOT NULL) OR (match_phone_digits IS NOT NULL) OR (match_name IS NOT NULL)`	`ck_blacklist_identifier`
Check	`(status = 'lifted') = (lifted_at IS NOT NULL)`	`ck_blacklist_lifted`
Check	`(effective_until IS NULL) OR (effective_until > effective_from)`	`ck_blacklist_period`
Check	`reason_category IN ('safety', 'theft', 'harassment', 'trespass', 'legal_order', 'policy_violation', 'other')`	`ck_blacklist_reason`
Check	`severity IN ('deny', 'require_approval')`	`ck_blacklist_severity`
Check	`status IN ('active', 'expired', 'lifted')`	`ck_blacklist_status`
Index	`btree (match_email) WHERE status = 'active'`	`ix_blacklist_email`
Index	`btree (person_id) WHERE status = 'active'`	`ix_blacklist_person`
Index	`btree (match_phone_digits) WHERE status = 'active'`	`ix_blacklist_phone`
Index	`btree (review_due_at) WHERE status = 'active'`	`ix_blacklist_review_due`
Trigger	`set_updated_at()`	`trg_blacklist_entries_updated_at`

Application rule. Matching visitors against these records at registration, approval and entry is intended to be handled by the application, which also limits the review date to 12 months. The reason is visible only to staff with blacklist.manage; reception and hosts see a neutral message.

2.28 blacklist_matches

SECURITY

PURPOSE Records each possible match between a visitor and a restricted-entry record, and the outcome of the human review of that match. A possible match is not treated as confirmed until someone authorised has reviewed it.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
blacklist_entry_id	UUID	FK → blacklist_entries *on delete restrict* Required	Record possibly matched.
visit_id	UUID	FK → visits *on delete restrict* Required	Visit screened.
match_basis	TEXT	Required	person, email, phone or name.
review_status	TEXT	Required · default `'pending'`	pending, confirmed or dismissed.
reviewed_by	UUID	FK → users *on delete restrict* Optional	Reviewer.
reviewed_at	TIMESTAMPTZ	Optional	When reviewed.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.

CONSTRAINT	DEFINITION	NAME
Unique	`blacklist_entry_id, visit_id`	`uq_blacklist_match_entry_visit`
Check	`match_basis IN ('person', 'email', 'phone', 'name')`	`ck_blacklist_match_basis`
Check	`(reviewed_by IS NULL) = (reviewed_at IS NULL)`	`ck_blacklist_match_review`
Check	`review_status IN ('pending', 'confirmed', 'dismissed')`	`ck_blacklist_match_review_status`
Check	`(review_status = 'pending') = (reviewed_by IS NULL)`	`ck_blacklist_match_reviewed`
Index	`btree (visit_id)`	`ix_blacklist_matches_visit`
Index	`btree (created_at) WHERE review_status = 'pending'`	`ix_blacklist_matches_pending`

Application rule. Screening again at check-in reuses the existing row for the same entry and visit. A name-only match is treated as weak and always needs review.

2.29 security_approvals

SECURITY

PURPOSE Records the security decision on whether a visitor held for review may proceed, and the reason for it. A held visit cannot be approved or checked in until this decision is made.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
visit_id	UUID	FK → visits *on delete restrict* Unique where status = 'pending' Required	Visit under review.
status	TEXT	Required · default `'pending'`	pending, approved, denied or superseded.
requested_at	TIMESTAMPTZ	Required · default `now()`	When review started.
decided_by	UUID	FK → users *on delete restrict* Optional	Decision maker.
decided_at	TIMESTAMPTZ	Optional	When decided.
decision_reason	VARCHAR(500)	Optional	Reason given.

CONSTRAINT	DEFINITION	NAME
Unique index	`visit_id WHERE status = 'pending'`	`uq_security_approval_pending`
Check	`status IN ('pending', 'superseded') OR ((decided_by IS NOT NULL) AND (decided_at IS NOT NULL) AND (decision_reason IS NOT NULL))`	`ck_security_approval_decision`
Check	`status IN ('pending', 'approved', 'denied', 'superseded')`	`ck_security_approval_status`
Index	`btree (status, requested_at)`	`ix_security_approvals_status`

Application rule. The application is intended to prevent a host from making the security decision for their own visitor. A normal visit approval never overrides a pending or denied security decision.

2.30 audit_logs

AUDIT

PURPOSE Keeps a lasting record of important actions, such as who approved a visit or changed an account. Entries cannot be changed through normal database access, so the record can be relied on in an investigation.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required	Unique identifier.
actor_user_id	UUID	FK → users *on delete restrict* Optional	User who performed the action, if any.
action	VARCHAR(80)	Required	e.g. visit.approved, blacklist.created.
created_at	TIMESTAMPTZ	Required · default `now()`	When the action occurred.
actor_device_id	UUID	FK → devices *on delete restrict* Optional	Device that performed it, if any.
entity_type	VARCHAR(40)	Required	Kind of record affected.
entity_id	UUID	Optional	Identifier of the affected record.
site_id	UUID	Optional	Site involved, if any.
correlation_id	VARCHAR(64)	Optional	Request id (X-Request-ID).
ip	INET	Optional	Client IP address.
metadata	JSONB	Required · default `'{}'`	Structured, non-sensitive details (JSON).

CONSTRAINT	DEFINITION	NAME
Index	`btree (action, created_at DESC)`	`ix_audit_logs_action`
Index	`btree (actor_user_id, created_at DESC)`	`ix_audit_logs_actor`
Index	`btree (entity_type, entity_id)`	`ix_audit_logs_entity`
Index	`btree (created_at DESC)`	`ix_audit_logs_created`
Index	`btree (site_id, created_at DESC) WHERE site_id IS NOT NULL`	`ix_audit_logs_site_created`
Trigger	`reject_history_change()`	`trg_audit_logs_append_only`

Application rule. Rows are intended to be written only through the application's audit service, which refuses anything that looks like a password, token or pass code. Metadata holds identifiers rather than personal details, so anonymising a person does not require editing the log.

2.31 notifications

AUDIT

PURPOSE Stores messages for staff, such as "your visitor has arrived", and tracks delivery of any email or text copy. It also lets each person see their own notifications in the dashboard; failed sends are retried without affecting the visit.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required	Unique identifier.
recipient_id	UUID	FK → users *on delete restrict* Required	Staff account notified.
visit_id	UUID	FK → visits *on delete set null* Optional	Visit concerned, if any.
title	VARCHAR(180)	Required	Title.
message	VARCHAR(500)	Required	Message text.
read	BOOLEAN	Required · default `false`	Whether the recipient has read it.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
kind	VARCHAR(40)	Required · default `'general'`	e.g. arrival, approval_needed, overdue.
channel	TEXT	Required · default `'in_app'`	in_app, email or sms.
status	TEXT	Required · default `'delivered'`	queued, attempting, accepted, delivered, failed or unknown.
attempt_count	INTEGER	Required · default `0`	Send attempts so far.
next_attempt_at	TIMESTAMPTZ	Optional	When the next send attempt is due.
last_error	VARCHAR(500)	Optional	Last delivery error.
provider_message_id	VARCHAR(200)	Optional	Delivery provider's message id.
read_at	TIMESTAMPTZ	Optional	When read.
delivered_at	TIMESTAMPTZ	Optional	When delivered.

CONSTRAINT	DEFINITION	NAME
Check	`channel IN ('in_app', 'email', 'sms')`	`ck_notifications_channel`
Check	`status IN ('queued', 'attempting', 'accepted', 'delivered', 'failed', 'unknown')`	`ck_notifications_status`
Check	`attempt_count ⩾ 0`	`ck_notifications_attempts`
Check	`status NOT IN ('queued', 'attempting') OR next_attempt_at IS NOT NULL`	`ck_notifications_due`

CONSTRAINT	DEFINITION	NAME
Index	`btree (recipient_id, read)`	`ix_notifications_recipient_read`
Index	`btree (next_attempt_at) WHERE status IN ('queued', 'attempting')`	`ix_notifications_due`

Application rule. A notification is intended to be written in the same transaction as the action that causes it; a failed send never undoes that action. Delivery, retries and repeated sends are described in Section 4.7.

2.32 retention_policies

OPS

PURPOSE Sets how long each kind of record is kept and whether it is deleted or anonymised afterwards. A system-wide default can be adjusted for individual sites, and each policy records who approved it.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
site_id	UUID	FK → sites *on delete restrict* Optional	Site the rule applies to; empty for the system-wide default.
record_class	TEXT	Required	ordinary_visit, security_visit, pass_scan, notification, auth_event, audit_log or job_run.
retain_months	INTEGER	Required	Months to keep.
action	TEXT	Required	delete or anonymize.
approved_by	UUID	FK → users *on delete restrict* Optional	Account that approved it; empty while it is a placeholder.
approved_at	TIMESTAMPTZ	Optional	When approved.
notes	VARCHAR(500)	Optional	Notes, e.g. the approving document.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Unique index	`site_id, record_class (nulls not distinct)`	`uq_retention_policy_scope`
Check	`action IN ('delete', 'anonymize')`	`ck_retention_action`
Check	`(retain_months ⩾ 1) AND (retain_months ⩽ 120)`	`ck_retention_months`
Check	`record_class IN ('ordinary_visit', 'security_visit', 'pass_scan', 'notification', 'auth_event', 'audit_log', 'job_run')`	`ck_retention_record_class`

CONSTRAINT	DEFINITION	NAME
Trigger	set_updated_at()	trg_retention_policies_updated_at

Application rule. The retention job is intended to apply the most specific rule (site, then system-wide) and to run only in preview mode for a record class until its policy has been approved (Section 5).

2.33 retention_holds

OPS

PURPOSE Marks a visit or a person whose records must be kept beyond the normal retention period, for example during a legal case or an investigation. Each hold needs a reason and a review date.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default gen_random_uuid()	Unique identifier.
visit_id	UUID	FK → visits *on delete restrict* Optional	Visit held, if the hold is on a visit.
person_id	UUID	FK → people *on delete restrict* Optional	Person held, if the hold is on a person.
hold_type	TEXT	Required	legal, investigation or security_incident.
reason	VARCHAR(500)	Required	Reason given.
case_reference	VARCHAR(120)	Optional	Case or matter reference.
placed_by	UUID	FK → users *on delete restrict* Required	Account that placed it.
placed_at	TIMESTAMPTZ	Required · default now()	When placed.
review_due_at	TIMESTAMPTZ	Required	Review date.
released_by	UUID	FK → users *on delete restrict* Optional	Account that released it.
released_at	TIMESTAMPTZ	Optional	When released.
release_reason	VARCHAR(500)	Optional	Reason given.

CONSTRAINT	DEFINITION	NAME
Check	(hold_type = 'security_incident') OR (case_reference IS NOT NULL)	ck_retention_hold_case_reference
Check	(released_at IS NULL) = (released_by IS NULL)	ck_retention_hold_release

CONSTRAINT	DEFINITION	NAME
Check	(review_due_at > placed_at) AND (review_due_at ⩽ (placed_at + '1 year'))	ck_retention_hold_review
Check	hold_type IN ('legal', 'investigation', 'security_incident')	ck_retention_hold_type
Check	num_nonnulls(visit_id, person_id) = 1	ck_retention_hold_target
Index	btree (visit_id) WHERE released_at IS NULL AND visit_id IS NOT NULL	ix_retention_holds_visit
Index	btree (person_id) WHERE released_at IS NULL AND person_id IS NOT NULL	ix_retention_holds_person

Application rule. While a hold is active, the retention job is intended to skip the visit or person and their related records (events, scans, matches and approvals). Review reminders are sent by the job.

2.34 job_runs

OPS

PURPOSE Records each run of a background task, such as the nightly clean-up, a database backup or a test restore, and whether it succeeded. This gives evidence that scheduled work, including backups, is actually happening.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
job_name	VARCHAR(60)	Required	e.g. retention, pending_expiry, backup, restore_test.
started_at	TIMESTAMPTZ	Required · default `now()`	Start.
finished_at	TIMESTAMPTZ	Optional	End.
status	TEXT	Required · default `'running'`	running, succeeded, failed or partial.
dry_run	BOOLEAN	Required · default `false`	Preview run whose changes were rolled back.
details	JSONB	Required · default `'{}'`	What the run did (JSON): counts, or backup location, size and checksum.
error	VARCHAR(1000)	Optional	Failure message.

CONSTRAINT	DEFINITION	NAME
Check	(status = 'running') = (finished_at IS NULL)	ck_job_runs_finished
Check	status IN ('running', 'succeeded', 'failed', 'partial')	ck_job_runs_status
Index	btree (job_name, started_at DESC)	ix_job_runs_name_started

> **Application rule.** Intended to be written by the background worker and the backup service. The backup files themselves, and their encryption keys, are kept outside the database.

3 Data types and key terms

The PostgreSQL types used in the proposed design, and the constraint terms used in the tables above.

Data types used

TYPE	COLUMNS	TYPICAL USE
`UUID`	100	Identifiers and references.
`TIMESTAMPTZ`	98	Dates and times, stored in UTC.
`VARCHAR`	57	Names, emails and labels with a length limit.
`TEXT`	36	Statuses, codes and free text without a length limit.
`INTEGER`	21	Counts, versions and settings.
`BOOLEAN`	12	True/false flags.
`JSONB`	7	Structured metadata and settings.
`SMALLINT`	5	Role and permission identifiers.
`INET`	4	IP addresses.
`BIGINT`	2	Sequential ids of append-only logs.

Key terms

TERM	MEANING
PK **Primary key**	Identifies one row uniquely.
FK **Foreign key**	Links a row to a record in another table; the database rejects links to missing rows.
Unique / unique index	Prevents duplicate values. A partial unique index applies only to rows matching its WHERE clause.
Check	A condition every row must satisfy.
Generated column	Computed by the database from other columns; cannot be written.
Restrict / cascade / set null	What happens to referencing rows when a referenced row is deleted: block the delete, delete them too, or clear the reference.
Trigger	Database code that runs on insert/update/delete (Section 5).

4 Security and reliability notes

4.1 Passwords and sign-in

TOPIC	PROPOSED APPROACH	ENFORCED BY
Storage	Hashes are kept in `user_credentials.password_hash`, separate from `users` and `people`.	Database (separate table)
Algorithm	Argon2id with a per-password salt, using a maintained library (pwdlib). Hashes made with outdated parameters are upgraded at the next sign-in.	Application
Plaintext guard	Check `ck_user_credentials_argon2id` rejects any value that does not start with `$argon2id$`.	Database
Reset links	Only a keyed hash of each reset token is stored (`password_reset_requests.token_hash`, unique), with at most one live request per account. Links go only to the login email.	Database (unique indexes); hashing by application
Sessions	Short-lived access tokens; each refresh token is random, stored only as a SHA-256 hash and replaced on every refresh. Revoking a session stops further refreshes.	Database (unique hash); rotation by application

4.2 QR pass tokens and other codes

No recoverable copy of a pass token is kept. The QR holds a random 256-bit token and no personal data. `passes.token_hash` stores its SHA-256 for lookup (unique); because the token is long and random, a plain SHA-256 is enough and no key is needed. A QR is shown, printed or sent when it is issued and cannot be displayed again from the database. Each visit has at most one pass (unique `visit_id`), and a pass has a single `token_hash`, so at most one token per pass is ever valid. Raw tokens are never written anywhere else either, including stored responses in `idempotency_keys`.

SECRET	STORED AS	WHY
QR pass token	SHA-256 (`passes.token_hash`)	High-entropy token; a database copy alone cannot be turned back into a working QR.
Device key and pairing code	Keyed hash with a server-side key (`devices`)	Pairing codes are short, so a key stops offline guessing.
Verification code	Keyed hash, short expiry and an attempt limit (`verification_codes`)	Codes of a few digits are guessable; the limit and expiry make guessing impractical.
Reset and refresh tokens	Hash only (Section 4.1)	Never needed again after they are sent.

4.3 Lost QR code or badge: two options

Without a recoverable token, a lost QR cannot simply be shown again. Two approaches are proposed for decision (Section 7). The schema supports both, and neither creates a second pass for the visit. In both, the visitor's identity is checked by staff first, and the visit keeps its status: a checked-in visitor stays checked in.

ASPECT	OPTION A: MANUAL VERIFICATION	OPTION B: TOKEN ROTATION ON THE EXISTING PASS
How it works	Staff revoke the lost pass and handle each remaining entry and exit by hand, checking the visitor's identity each time. No new QR is issued.	The existing pass is given a new token. The old QR stops working, and only the new one, shown or printed once, can be used from then on.
The old QR, if found and used	Refused as revoked; the attempt is recorded against the visit in `pass_scans`.	Refused as an unknown code. The attempt cannot be linked to the visit, because the old hash is no longer stored.
Reception workload	Every remaining entry and exit needs a staff member. This weighs most on visits lasting several days and on busy entrances; the kiosk cannot be used.	One staff interaction; afterwards the visitor uses the kiosk as normal.
Security	No new credential is created. Protection depends on consistent identity checks at every manual entry.	The new QR must reach only the verified visitor. Frequent reissues for one visit should be visible to staff, as they may point to misuse.
Atomicity	Revoking the pass and recording the event happen in one transaction.	The pass row is locked, `token_hash` is replaced and the `pass_reissued` event written in one transaction; the new QR is shown only after the commit. If the transaction fails, the old token simply stays valid; the two can never be valid together.
What is recorded	The revocation (`revoked_by`, `revoke_reason`) and every manual entry and exit, each naming the staff member who checked the visitor.	A `pass_reissued` event naming the staff member and the reason, plus an audit entry.
Schema use	Existing columns only; `pass.reissue` and the `pass_reissued` event type go unused.	Existing columns, the `pass.reissue` permission and the `pass_reissued` event type.

Neither option can reprint the same QR, so a damaged badge is handled like a lost one unless the visitor still holds the original QR, for example in their email. Under Option B, if later use of a replaced QR had to be traced, the previous token's hash could be kept for recognition only; that is not part of this proposal.

4.4 Screening and admission decisions

Five separate facts are recorded, so a possible match is never confused with a confirmed one, and a confirmed match is never confused with the admission decision.

STEP	RECORDED IN	NOTES
Restriction	`blacklist_entries`	Reason (restricted), severity, responsible person, end date and review date.
Possible match	`blacklist_matches`, review_status `pending`	One row per entry and visit; `match_basis` shows how strong it is. Not treated as a confirmed restriction.

STEP	RECORDED IN	NOTES
Review of the match	`blacklist_matches`, `confirmed` or `dismissed`, with reviewer	Name-only matches always need review.
Admission decision	`security_approvals`, `approved` or `denied`, with decider and reason	The database refuses to schedule or admit the visit while the review is pending or denied, so an ordinary approval cannot override it. Never made by the visitor's own host, and a confirmed match on a `deny` entry cannot be approved (both enforced by the application).
Access attempt	`pass_scans`, outcome `blacklisted` or `security_pending`	Screening is repeated at each entry, so a restriction added during a longer visit stops re-entry. Refused attempts are recorded as well as accepted ones.

4.5 Protecting history and attributing actions

PROTECTION	ENFORCED BY
`audit_logs`, `visit_events`, `authentication_events`, `pass_scans` and `consent_notices` are append-only.	Database: trigger, plus the application's database role has no UPDATE, DELETE or TRUNCATE rights on them (Section 6)
Approvals, rejections, exceptions, extensions, host changes, manual corrections and pass reissues name the staff member; all but approvals, rejections and extensions also need a reason.	Database: `ck_visit_events_actor`, `ck_visit_events_reason`
Every entry and exit comes from a registered device or names a staff member.	Database: `ck_visit_events_movement`
Alternative identification names the receptionist, and a walk-in cannot be scheduled without identification.	Database: `ck_visits_verified_by`, `ck_visits_walk_in_verified`
Extensions record who asked, who decided, when, and the old and new end times.	Database: `visit_extensions` keys and checks
Security decisions name the decider and a reason.	Database: `ck_security_approval_decision`
An approval by anyone other than the visit's host carries a reason.	Application
Every administrative change is written to `audit_logs` in the same transaction.	Application
No passwords, tokens or unnecessary personal details in audit metadata.	Application (audit service)

4.6 Backups and keys

ITEM	WHERE IT LIVES	NOTES
Backup files	Outside the database, encrypted, with a copy off the database host	Kept for their own, separately agreed period; the retention job does not trim them.

ITEM	WHERE IT LIVES	NOTES
Backup encryption key	A secrets manager or offline escrow; never in this database and never stored with the backup files	Needed for every restore, so who holds it and how it is rotated must be planned.
Application keys	Application secret store	Token-signing key and the server-side key for device keys, reset tokens and verification codes.
Backup records	`job_runs` (job `backup` or `restore_test`)	A convenience record of location, size and checksum. The backup service's own logs remain authoritative, as this table is lost with the database.
Restores	Into a scratch database first	Restored data goes through the retention job before use, so records removed since the backup do not reappear. Recovery procedures are operational and cannot be guaranteed by the schema.

4.7 Notification delivery and kiosk data refresh

TOPIC	PROPOSED APPROACH	ENFORCED BY
Writing	A notification is written in the same transaction as the change that causes it, so a rolled-back change sends nothing and a failed send never undoes the change.	Database transaction
Claiming	A worker takes due rows (`queued` or `attempting`, `next_attempt_at` passed), skipping rows another worker has locked. It marks them `attempting` and moves `next_attempt_at` forward as a lease, so rows held by a worker that crashes become due again.	Application; database check that a queued row always has a due time
Retrying	Failures increase `attempt_count`, keep `last_error` and are retried with a growing delay; after an agreed number of attempts the status becomes `failed` and staff can see it.	Application
Repeated sends	If a worker stops after the provider accepted a message but before recording it, the message may be sent twice. The notification id is passed to the provider as an idempotency key where supported, and an unclear outcome is recorded as `unknown`.	Application
Kiosk refresh	The server stays the authority; a kiosk's copy of the day's visits is a convenience. Changes are fetched by (`updated_at`, `id`) with an overlap window, because timestamps alone can miss a row from a slow transaction that commits after a later one has been read. The kiosk keeps a row only if its `version` is newer, and refetches the full day's list periodically and after reconnecting. Visits are never hard-deleted, so removals need no tracking.	Application
Other systems	No change feed is proposed until a downstream system and its needs are known; an outbox or change-data capture can be added then.	-

5 Retention, anonymisation and legal holds

Retention periods are an organisational policy owned by the privacy-notice owner, not a property of the schema. The design therefore stores them as data that can be approved and changed, and the proposed structure does not depend on the exact values.

5.1 Configurable policies

PRINCIPLE	HOW THE DESIGN SUPPORTS IT
Periods are policy, not structure	Each period is a row in `retention_policies`; changing it needs no schema change.
Different data, different rules	One policy per record class, so ordinary visits, security-related visits, audit logs and operational logs can each have their own period and action.
Sites may differ	A site row overrides the system-wide default for that class.
Decisions are accountable	`approved_by` and `approved_at` record who approved each policy, and changes are audited. Until a policy is approved, the retention job only previews its effect.
Effects can be checked first	A preview run (`job_runs.dry_run`) reports what would be deleted or anonymised without changing anything.

5.2 Placeholder values

The values below are placeholders for discussion. Neither the five-month nor the 24-month period has been approved or confirmed as legally required, and they are not assumed to apply to the same data. Months are calendar months counted from when a record closes (`visits.closed_at` for visits).

RECORD CLASS	COVERS	PLACEHOLDER	ACTION	BASIS
`audit_log`	audit_logs	24 months	delete	Working assumption, as for security_visit.
`auth_event`	authentication_events	12 months	delete	Working assumption.
`job_run`	job_runs	6 months	delete	Working assumption.
`notification`	notifications	3 months	delete	Working assumption; also removed when the related visit is anonymised.
`ordinary_visit`	Closed visits with no security involvement, with their events	5 months	anonymize	A period of five calendar months has been discussed for ordinary completed visits.
`pass_scan`	pass_scans	5 months	delete	Working assumption, in line with ordinary visits.
`security_visit`	Visits that had a possible match or security review, with their matches and decisions	24 months	anonymize	Working assumption. A 24-month period has been discussed, but which data it should cover is not yet clarified.

Not covered by a class: restriction records in `blacklist_entries`, which have their own end and review dates (how long they are kept afterwards is an open decision); `idempotency_keys` and `verification_codes`, which the clean-up job removes about a day after expiry; and staff accounts.

5.3 Anonymisation

For visits, anonymisation is preferred to deletion: the row and its links stay, so history and reports on past visit numbers remain consistent, while personal details are removed from the visit and from every related record that can hold them.

RECORD	REMOVED	KEPT
`people`	Name replaced by a placeholder; email, phone and company cleared, including on records merged into this one. Done only when none of the person's visits is still within its period or on hold, and no active restriction names them.	The row, so visits keep a valid link
`visitor_profiles`	The whole row	-
`visits`	Purpose; link to anyone who registered on the visitor's behalf	Site, host, times, statuses, source, category and verification method
`visit_events`	Reason and metadata	Event types, times and staff actors
`visit_extensions`	Reason and decision note	Times, status and staff actors
`passes`	Revocation reason	Status and validity (the token hash is of a random value)
`security_approvals`	Decision reason	Status, decider and times
`person_merge_log`	Reason	Which records were merged, by whom
`notifications`	Rows linked to the visit, at the latest when it is anonymised, whatever their own policy	-
`pass_scans`, `verification_codes`, `idempotency_keys`	Removed earlier under their own, shorter rules	-

5.4 Legal holds

RULE	ENFORCED BY
An active hold pauses both deletion and anonymisation. A hold on a visit covers its events, extensions, scans, matches, approvals and notifications; a hold on a person also covers all of their visits.	Application (retention job)

RULE	ENFORCED BY
The retention clock is not reset: once a hold is released, anything past its period is processed at the next run.	Application (retention job)
Each hold needs a reason and a review date within a year; legal and investigation holds need a case reference.	Database checks
Placing and releasing a hold are audited, with the person responsible.	Database columns; audit by application
Holds and retention do not change backups, which expire on their own schedule (Section 4.6).	Operational policy

6 Database functions, access and seed data

Rules the database itself is intended to enforce beyond keys and checks, the database roles that make those rules hold, how staff access is decided, and the reference data to be loaded when the database is created.

FUNCTION	ATTACHED TO	WHAT IT ENFORCES
enforce_user_role_scope()	user_roles	Rejects a global role assigned to a site, and a site-level role assigned without a site (based on roles.scope).
reject_history_change()	audit_logs, authentication_events, consent_notices, pass_scans, visit_events	Makes the table append-only: UPDATE and DELETE fail unless the current database role is vms_maintenance, which only the retention job uses.
reject_inactive_host()	visits	Rejects a new visit, or a change of host, for a host whose status is not active.
reject_uncleared_visit()	visits	Rejects changing a visit to scheduled or checked-in while its current security review is pending or denied. Leaving the site is never blocked.
set_updated_at()	blacklist_entries, departments, devices, gates, hosts, passes, people, retention_policies, site_settings, sites, user_credentials, users, visitor_profiles, visits	Sets updated_at to now() on every update.

Extensions: pg_trgm (trigram indexes for name and employee-ID search, and similarity matching), plus plpgsql.

Database roles

Separate database logins, so that the protections above do not depend on the application behaving correctly.

ROLE	USED BY	CAN DO
`vms_owner`	Migrations only	Create and change tables, functions and grants. Not used by the running application.
`vms_app`	API and background worker	Read and write operational tables; insert into the append-only tables but never update, delete or truncate them.
`vms_maintenance`	Retention job	Delete or anonymise records past their retention period, including history tables. Each run is recorded in `job_runs`.
`vms_backup`	Backup service	Read-only access for taking backups.

How staff access is decided

Role permissions say what kind of action a staff member may take; record-level rules say which records they may take it on. Both must pass.

LAYER	RULE	ENFORCED BY
Role permissions	An action needs a permission granted by one of the staff member's roles; several roles combine.	Application, using `role_permissions`
Site scope	Only roles that cover the site of the record count; a role with no site covers every site. A host role at one site and a receptionist role at another never combine into receptionist rights at the first.	Application; scope shape by `enforce_user_role_scope`
Record rules	Hosts see and act on visits where they are the host; reception and site administrators act on visits at their sites; restriction reasons need `blacklist.manage`; a host never decides the security review of their own visitor.	Application
Devices	A kiosk acts only for visits at its own site.	Application
Changes	Removing a role or a site takes effect at the next token refresh at the latest; disabling an account revokes its sessions.	Application

Proposed staff roles

A global role applies to every site; `global_or_site` roles can cover every site or selected sites; site roles always name their sites. A staff member may hold several roles; at any site, their permissions are those of the roles that cover it.

CODE	SCOPE
`super-admin`	global
`site-admin`	global_or_site
`receptionist`	site
`host`	site

Proposed permissions

Grants that depend on an open decision are listed separately and are not part of the proposed defaults.

PERMISSION	PROPOSED	DEPENDS ON A DECISION
`system.manage`	super-admin	-
`site.manage`	super-admin, site-admin	-
`user.read`	super-admin, site-admin	-
`user.manage`	super-admin, site-admin	-
`user.reset_password`	super-admin, site-admin	-
`host.read`	super-admin, site-admin, receptionist, host	-
`host.manage`	super-admin, site-admin	-
`device.manage`	super-admin, site-admin	-
`visit.read`	super-admin, site-admin, receptionist, host	-
`visit.create`	super-admin, site-admin, host	-
`visit.update`	super-admin, site-admin, host	-
`visit.approve`	super-admin, site-admin, host	receptionist, for walk-ins in defined circumstances (decision 03)
`visit.reject`	super-admin, site-admin, host	-
`visit.cancel`	super-admin, site-admin, receptionist, host	-
`visit.extension.request`	-	host, receptionist (decision 05)
`visit.extension.approve`	super-admin, site-admin	host, receptionist (decision 05)
`visit.override`	super-admin, site-admin	-
`walk_in.register`	super-admin, site-admin, receptionist	-
`visit.check_in`	super-admin, site-admin, receptionist	-
`pass.revoke`	super-admin, site-admin, host	-
`pass.reissue`	-	super-admin, site-admin, receptionist, only if Option B is chosen (decision 01)
`visitor.update`	super-admin, site-admin, receptionist	-
`visitor.merge`	super-admin, site-admin	-
`blacklist.manage`	super-admin, site-admin	-
`security.approve`	super-admin, site-admin	-
`audit.read`	super-admin, site-admin	-
`retention.hold`	super-admin, site-admin	-

7 Decisions to confirm

Business and operational decisions that affect parts of this design. Where a default is needed, the schema supports either outcome; none of these is treated as decided.

#	TOPIC	QUESTION
01	**Lost QR or badge**	Option A (manual verification by staff) or Option B (a new token on the existing pass)? See Section 4.3.
02	**Retention periods**	Confirm the period for each record class, what the 24-month value applies to, how long restriction records are kept after they end, and who approves each policy.
03	**Walk-in decisions by reception**	In which circumstances may an authorised receptionist decide a walk-in instead of the host, and which receptionists are authorised?
04	**Alternative identification**	Which alternatives are approved when OTP cannot be completed (reception ID check, a reception-issued code, or both), and which IDs are acceptable? Which OTP channels are used: text message, email or both?
05	**Extensions**	Who may request an extension and who may approve one? May the same person do both?
06	**Delegated approval**	May someone approve on behalf of an absent host? If so, a small host-delegate table would be added.
07	**Visiting hours**	At which sites are visiting hours enforced, and who may grant an exception?
08	**Hosts at several sites**	Can a host receive visitors at sites other than their home site?
09	**Security reviewers**	Are security decisions made by site administrators, or by a separate security role?
10	**Offline kiosk**	Must the kiosk work without a connection? If so, how should check-ins made offline be confirmed afterwards?
11	**Notification channels**	Which channels do staff notifications need in the first release: in-app and email only, or text messages as well?
12	**Group visits**	Can one request cover several visitors? If so, a parent table for the shared request would be added above visits.
13	**Extra identifiers**	Should screening use further identifiers, such as an ID number, given the privacy cost of storing them?

SUGGESTED NEXT STEP

Review the proposed tables and lifecycle with the team, settle the decisions above, and agree the retention periods with the privacy-notice owner before the schema is settled.